2025 10th International Conference on Computer Science and Engineering (UBMK) | 979-8-3315-9975-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/UBMK67458.2025.11206755

AutoStructor: A Generative AI-Based Framework for Automated Program Repair with Deep Learning-Guided Fault Localization

Hasen Özaytürk
Department of Computer Engineering
Istanbul Technical University
Istanbul, Türkiye
ozayturk22@itu.edu.tr

Feza Buzluca
Department of Computer Engineering
Istanbul Technical University
Istanbul, Türkiye
buzluca@itu.edu.tr

***Abstract*—Current automated program repair (APR) approaches still suffer from two critical limitations: inaccurate fault localization and ineffective patch generation. This paper presents a novel hybrid framework, AutoStructor, that proposes a Quality-Aware Graph-Based Machine Learning Fault Localization (GBMFL) and a Large Language Model (LLM)-based patch generation technique to effectively address these challenges. Our key innovation lies in integrating 21 comprehensive code quality metrics and sophisticated multi-modal attention mechanisms, including quality-aware, graph-aware, sparse, and multiscale attention into the GBMFL architecture, coupled with intelligent test failure analysis for superior code understanding and patch generation. AutoStructor bridges the gap between fault localization and patch generation through an end-to-end pipeline that leverages generative AI for intelligent patch synthesis. We evaluate our approach on the Defects4J benchmark, demonstrating significant improvements in fault localization accuracy (81.9% Top-1, 93.4% Top-3) with a Mean First Rank of 1.13, and successful patch generation for 48 out of 61 active Lang project bugs. Notably, our structured test analysis approach enables cost-effective deployment using compact LLMs, achieving effective repair.**

***Keywords*—automated program repair, graph-based fault localization, multi-modal attention, test failure analysis, software quality metrics**

I. Introduction

Software bugs impose significant costs on development teams and end users. Traditional debugging approaches rely heavily on manual inspection, making the process time-consuming and error-prone. Automated Program Repair (APR) has emerged as a promising solution; however, current approaches face fundamental limitations that hinder its widespread adoption.

Spectrum-Based Fault Localization (SBFL) techniques rely primarily on code coverage information while failing to capture underlying code quality characteristics that correlate with fault-proneness. Existing neural fault localization approaches focus on structural dependencies while neglecting the multi-dimensional nature of code quality. Most critically, existing APR approaches treat fault localization and patch generation as separate, loosely coupled phases, resulting in suboptimal results as patch generation fails to leverage the rich contextual information gathered during localization.

This paper makes the following key contributions to address these gaps:

- **Quality-Aware GBMFL Framework with Neural Architecture**: We present a novel approach that integrates comprehensive code quality metrics into Graph-Based Machine Learning Fault Localization. Our quality-aware framework incorporates 21 quality metrics across theoretically-grounded categories, including complexity, coupling, cohesion and testability.
- **Multi-Modal Attention Mechanisms for Advanced Code Understanding**: We introduce four specialized attention mechanisms designed specifically for code analysis: (a) Quality-aware attention with relative positioning and adaptive temperature scaling for sequence understanding, (b) Graph-aware attention that explicitly models program structure through edge types and connectivity patterns, (c) Sparse attention for computational efficiency while maintaining performance through importance-based token selection, and (d) Multi-scale attention for hierarchical feature extraction at different granularities.
- **Intelligent Failed Test Analysis for APR**: We introduce LLM-based test failure explanation that systematically bridges fault localization and patch generation by extracting structured insights from test failures, stack traces, and assertion errors to guide repair synthesis.
- **End-to-End LLM-Guided APR Pipeline**: We develop a comprehensive framework that seamlessly integrates deep learning fault localization with generative AI patch generation.
- **Experimental Evaluation and Analysis**: We provide an extensive evaluation on the Defects4J benchmark with a detailed analysis of both fault localization performance and repair success.

II. Related Work

A. Fault Localization Techniques

Recent advances have demonstrated remarkable performance improvements through the integration of deep learning. DeepFL pioneered multi-dimensional fault localization

979-8-3315-9975-1/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on May 22,2026 at 23:53:41 UTC from IEEE Xplore. Restrictions apply.

by integrating spectrum-based, mutation-based, complexity-based, and textual-similarity features, localizing over 50 more faults within Top-1 compared to traditional approaches [1]. GRACE introduced graph neural networks to fault localization, achieving 195 bug localizations within the Top-1 category, compared to 166 for the best baseline [2]. DEEPRL4FL treated fault localization as image pattern recognition, achieving 173.1% to 491.7% improvement over statement-level baselines [3]. Traditional SBFL techniques, such as Tarantula [4] and Ochiai [5], have established foundational approaches but suffer from limited precision in complex scenarios.

The recent work by Rafi et al. [6] introduces Dependency-Enhanced Coverage Graph (DepGraph) that integrates inter-procedural call graph information with traditional AST-based representations, achieving 20% improvement in Top-1 accuracy over GRACE while reducing computational overhead by 70%. This work demonstrates the importance of incorporating method-level call dependencies for more accurate fault localization.

However, existing approaches face critical limitations. Traditional SBFL methods suffer from limited precision in complex fault scenarios. Neural approaches typically focus on single-dimensional features while neglecting the multi-dimensional nature of code quality metrics that correlate strongly with fault-proneness.

B. Automated Program Repair

The field has evolved through several generations, with recent LLM-based approaches showing remarkable capabilities. Traditional template-based repair approaches, such as PAR [7], rely on predefined fix templates but are limited by template coverage. Neural program repair approaches like SequenceR employ neural machine translation techniques [8], while CURE combines neural translation with programming language models, fixing 62 bugs on Defects4J [9].

Recent LLM-based advances have revolutionized automated program repair. ThinkRepair leverages Chain-of-Thought prompting for self-directed repair, fixing 98 bugs on Defects4J V1.2 with 27%-344.4% improvement over baselines [10]. LLMAO introduces test-free fault localization using bidirectional adapter layers on frozen LLMs, achieving 2.3%-54.4% Top-1 improvement [11].

Despite these advances, most existing approaches treat fault localization and patch generation as separate phases. This separation often leads to suboptimal results, as patch generation strategies fail to leverage the rich contextual information gathered during fault localization.

C. Code Quality Metrics and Fault Prediction

The relationship between code quality metrics and fault-proneness has been extensively studied. McCabe established cyclomatic complexity with threshold categories: 1-10 (simple), 11-20 (moderate), 21-50 (complex), and above 50 (very complex) [12]. The Chidamber-Kemerer (CK) metrics suite provides comprehensive object-oriented measurement [13], with Basili et al. validating that CK metrics significantly correlate with fault-proneness [14].

Recent empirical studies demonstrate that methods with cyclomatic complexity greater than 10 exhibit a 3.2 times higher fault probability, while cognitive complexity above 15 shows a 4.1 times higher fault rate. Learning-based approaches have shown that design quality metrics can be effectively used for fault detection in identifying error-prone code elements [15]. Our work is the first to systematically integrate these comprehensive quality dimensions into Graph-Based Machine Learning Fault Localization architectures.

III. AutoStructor Framework

A. System Architecture Overview

AutoStructor presents a comprehensive end-to-end pipeline that seamlessly integrates Quality-Aware Graph-Based Machine Learning Fault Localization with Large Language Model-based patch generation. The framework architecture comprises three primary modules: Fault Localization, Test Explainer, and Patch Generation. These modules work in a coordinated manner to ensure accurate fault identification and efficient patch synthesis. This end-to-end interaction is illustrated in Figure 1.

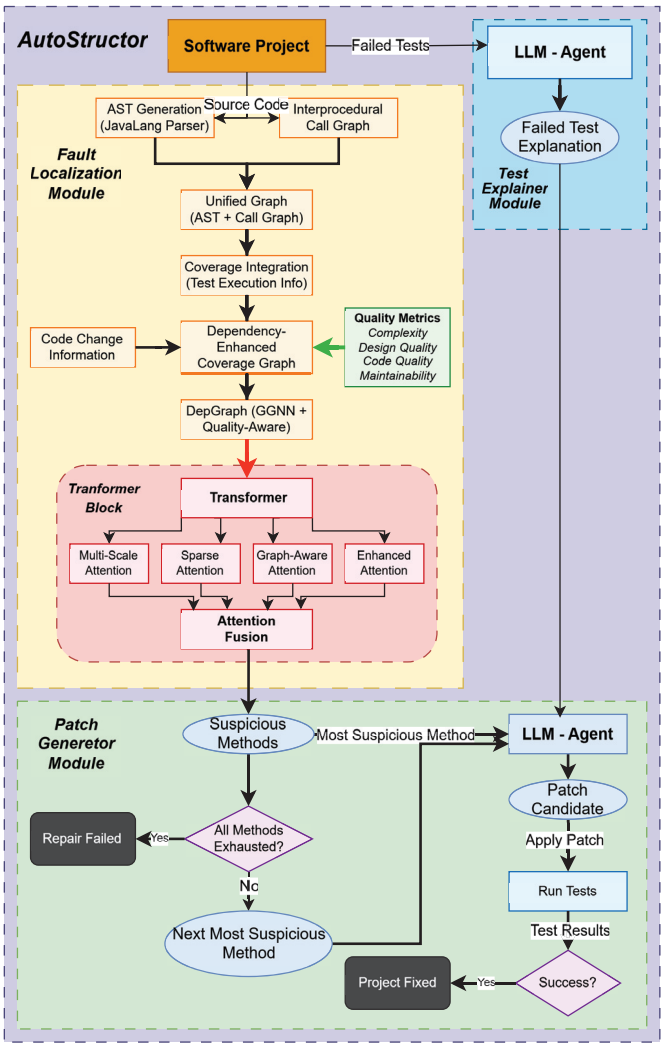


Fig. 1: AutoStructor End-to-End Pipeline Architecture

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on May 22,2026 at 23:53:41 UTC from IEEE Xplore. Restrictions apply.

B. End-to-End Repair Process

AutoStructor implements a four-phase repair process:

Phase 1: Project Setup and Environment Initialization: The framework checks out the target buggy version from repository, creates isolated workspace environments, and establishes proper compilation and testing environments.

Phase 2: Quality-Aware Fault Localization: Our *Fault Localization Module* adapts the DepGraph architecture from [6] as the foundation and enhances it with our novel quality-aware extensions. We integrate 21 quality metrics across complexity, design, code quality, and maintainability dimensions into the baseline graph structure. The module employs a hybrid attention mechanism incorporating four specialized attention types: quality-aware, graph-aware, sparse, and multi-scale attention for superior code understanding and fault ranking.

Phase 3: Test Analysis and Context Generation: The *Test Explainer Module* analyzes failing test cases to extract meaningful insights about fault characteristics and expected behavior using lightweight LLM analysis to interpret test failure messages, stack traces, and assertion failures.

Phase 4: Iterative Patch Generation and Validation: The *Patch Generator Module* systematically attempts repair on ranked suspicious methods through multiple iterations, applying advanced prompt engineering, and validating generated patches through automated test execution.

While our framework adapts the DepGraph architecture [6] as the foundational fault localization component, our key innovations include: Integration of 21 comprehensive quality metrics beyond traditional coverage-based features, hybrid multi-modal attention mechanisms specifically designed for quality-aware code analysis and intelligent test failure explanation that bridges fault localization with patch generation through structured LLM analysis.

C. Fault Localization Module

1) Quality Metrics Framework

Our quality-aware GBMFL framework incorporates 21 carefully selected quality metrics organized into four theoretically-grounded categories:

Complexity Metrics capture the intrinsic difficulty of understanding and maintaining code segments. Cyclomatic and cognitive complexity measure code understanding difficulty, while nesting depth, method length, parameter count, and return complexity assess structural complexity aspects, comprising 6 metrics in total.

Design Quality Metrics assess adherence to fundamental software engineering principles. The Single Responsibility Principle (SRP) score quantifies the extent to which a method focuses on a single concern through domain-specific responsibility analysis. Coupling metrics quantify inter-module dependencies through analysis of import statements, object instantiations, and method call patterns. Additional metrics include cohesion score, abstraction level, and design pattern usage detection, totaling 6 metrics.

Code Quality Metrics assess surface-level characteristics that impact readability and maintainability. Naming quality evaluation incorporates multiple heuristics including length appropriateness, camelCase convention adherence, semantic meaningfulness, and appropriate verb usage for behavioral methods. This category also includes comment ratio, code smells detection, duplication analysis, and magic numbers identification, encompassing 5 metrics.

Maintainability Metrics predict long-term code evolution costs through assessment of testability, reusability, and error handling quality. Testability evaluation combines multiple factors including method visibility, parameter complexity, static dependency minimization, and correlation with single responsibility adherence. This category encompasses readability score, testability score, reusability score, and error handling quality assessment, consisting of 4 metrics.

Table I provides the complete specification of our 21 quality metrics organized by category.

More detailed specifications and thresholds for all 21 quality metrics are available in our implementation repository [16].

2) Multi-Modal Attention Mechanisms

We introduce four specialized attention mechanisms designed specifically for code analysis:

Quality-Aware Attention with Relative Positioning addresses the limitation of traditional attention mechanisms through relative positional encodings, enabling the model to understand sequential relationships between code elements with adaptive temperature scaling for improved convergence.

Graph-Aware Attention for Structural Understanding explicitly models program structure through edge types and connectivity patterns. This mechanism incorporates edge type embeddings for different relationship types (control flow, data flow, call dependencies) and implements structure encoding through neighborhood aggregation.

Sparse Attention for Computational Efficiency reduces computational complexity while maintaining performance through importance-based token selection. Our implementation employs a learned importance scoring network that evaluates pairwise relationships between code elements.

Multi-Scale Attention for Hierarchical Feature Extraction

TABLE I: 21 CODE QUALITY METRICS

Category	Metric	Count
Complexity	Cyclomatic Complexity, Cognitive Complexity, Nesting Depth, Method Length, Parameter Count, Return Complexity	6
Design Quality	SRP Score, Afferent Coupling, Efferent Coupling, Cohesion Score, Abstraction Level, Pattern Usage	6
Code Quality	Naming Quality, Comment Ratio, Code Smells, Duplication Score, Magic Numbers	5
Maintainability	Readability Score, Testability Score, Reusability Score, Error Handling	4

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on May 22,2026 at 23:53:41 UTC from IEEE Xplore. Restrictions apply.

captures features at different granularities, enabling both local and global pattern recognition through multiple attention heads operating at different scales with scale-specific projections and learnable fusion weights.

We enhance the baseline DepGraph suspicion scores by integrating 21 code quality metrics through a neural gated fusion mechanism, enabling dynamic weighting based on method characteristics and quality patterns.

Despite the integration of quality metrics and multi-modal attention mechanisms, our approach maintains computational efficiency comparable to the baseline DepGraph architecture through optimized metric computation and sparse attention mechanisms.

D. Test Explainer Module: Failed Test Analysis

The Test Analysis Module enhances the patch generation process by extracting structured explanations from failing test cases. Using large language model analysis, this component processes test failure outputs, stack traces, and assertion failures to generate concise contextual descriptions that capture what went wrong and why the test failed.

Our approach employs structured prompting to guide the large language model in extracting essential test semantics and failure characteristics. The analysis focuses on three key aspects: the functionality being validated by the test, the type of bug suggested by the failure pattern, and the expected correct behavior. Rather than having the patch generation process deal solely with error messages or test source code alone, this approach provides meaningful context to the patch generator by explaining what the test means in a few key sentences.

The structured prompting approach effectively extracts test semantics and failure characteristics. These explanations provide additional context to the patch generator and aim to improve repair quality.

E. Patch Generator Module

The LLM-based patch generation module transforms identified suspicious code regions into concrete repair actions through sophisticated integration with modern generative AI capabilities.

Suspicious method ranking and selection strategy leverages fault localization scores to prioritize patch generation efforts on the most promising candidates. The selection process considers not only suspicion scores but also practical factors such as method complexity, availability of source code, and likelihood of successful patch application.

Test-aware context generation leverages the structured test failure explanations from the Test Analysis Module to provide LLMs with focused contextual understanding about intended behavior and fault characteristics. This approach enables more targeted patch generation by informing the model about the correct functionality that failing tests validate.

Iterative repair process executes patch generation systematically for each suspicious method, validating candidates through automated test execution before proceeding to the next method. The framework employs multi-candidate generation, producing alternative patch formulations when initial attempts fail to pass test validation.

Prompt engineering for effective patch generation implements sophisticated strategies for providing large language models with appropriate context for patch generation, incorporating both source code analysis and test failure insights. Our prompts carefully balance specificity with generality, providing sufficient context about fault characteristics while avoiding overly constraining the solution space.

IV. Experimental Evaluation

A. Experimental Setup

Dataset and Benchmark Selection: We conduct our primary evaluation on the Defects4J benchmark [17], focusing on the Lang project which contains 65 bugs with well-documented fault locations and comprehensive test suites. We exclude deprecated bugs (Lang-2, Lang-18, Lang-25, and Lang-48), resulting in 61 active bugs for comprehensive analysis.

Baseline Comparisons: We compare our quality-aware GBMFL approach against the baseline proposed in [6], as the GBMFL baseline already surpasses traditional SBFL methods and neural fault localization models.

Evaluation Metrics: We employ standard fault localization metrics including Top-N accuracy, representing the percentage of bugs where the faulty method appears in the top N ranked results, and Mean First Rank (MFR), representing the average rank of the first correctly identified faulty method.

LLM Implementation Details: Our framework employs separate GPT-4o Mini instances for test failure analysis and patch generation components to ensure proper isolation. Each bug repair session initializes fresh LLM contexts excluding conversation history to prevent information leakage between different repair attempts. This design ensures that each patch generation is based solely on the current bug's context and fault localization results, maintaining experimental integrity across all evaluations.

B. Fault Localization Results

Our quality-aware GBMFL approach demonstrates significant improvements in fault localization accuracy compared to baseline approaches. We re-implement and evaluate the baseline DepGraph method [6] in our experimental environment to ensure fair comparison under identical conditions.

Table II presents a progressive evaluation on the Defects4J Lang project, demonstrating the cumulative effectiveness of our quality-aware enhancements. Starting from baseline DepGraph [6], we incrementally add multi-modal attention, quality metrics, and optimization to achieve our final AutoStructor configuration.

Our final optimized implementation achieves the best

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on May 22,2026 at 23:53:41 UTC from IEEE Xplore. Restrictions apply.

TABLE II: FAULT LOCALIZATION PERFORMANCE COMPRASION

Method	Top-1	Top-3	Top-5	Top-10	MFR
Baseline GBMFL [6]	45	56	57	61	1.27
Multi-Modal Attention	45	56	59	61	1.39
Quality Metrics	46	55	60	61	1.45
AutoStructor	**50**	**57**	**60**	**61**	**1.13**

performance with 50 Top-1 accuracy (81.9%), 57 Top-3 accuracy (93.4%), 60 Top-5 accuracy (98.4%), and 61 Top-10 accuracy (100%). Most significantly, the MFR improves to 1.13, matching DepGraph's computational efficiency while incorporating comprehensive quality assessment.

C. End-to-End Automated Program Repair Results

Building on the demonstrated fault localization improvements in Section IV.B, we evaluate the complete AutoStructor framework for end-to-end program repair.

Overall Repair Success Rates: Among the 61 active Lang project bugs, AutoStructor framework successfully generates and validates patches for 48 bugs, achieving an overall success rate of 78.7%.

Comparison with State-of-the-Art LLM-based APR Tools: Recent LLM-based APR tools have shown significant improvements on the Defects4J benchmark. Table III provides detailed comparison with state-of-the-art approaches.

AutoStructor achieves 48 correct fixes, representing a 50% improvement over the best existing LLM-based tool (ChatRepair with 32 fixes) and demonstrating substantial superiority over recent approaches.

D. Case Study: Successful Repair Examples

We demonstrate the practical effectiveness of our end-to-end automated program repair approach through detailed analysis of two successful repair cases from the Defects4J Lang project.

1) Lang-14: CharSequence Comparison Issue

Bug Description: Lang-14 involves a fault in the `StringUtils.equals` method that fails to properly handle `CharSequence` comparison, leading to test failures in `StringUtilsEqualsIndexOfTest`.

Fault Localization Results: Our quality-aware GBMFL identified the `equals` method as the primary suspicious target with a high confidence score, ranking it among the top candidates for repair.

Figure 2 shows the original buggy implementation that fails to handle CharSequence comparison properly.

TABLE III: COMPARISON WITH RECENT LLM-BASED APR TOOLS

Tool	Lang Project Fixes
ITER [18]	17
SelfAPR [18]	21
AlphaRepair [19]	27
Repilot [19]	29
FitRepair [19]	29
ChatRepair [19]	32
AutoStructor	**48**

```java
public static boolean equals(CharSequence cs1,
    CharSequence cs2) {
   if (cs1 == cs2) {
       return true;
   }
   if (cs1 == null || cs2 == null) {
       return false;
   }
   return cs1.equals(cs2);
}
```

Fig. 2: Original Buggy Implementation of Lang-14

The issue was that `CharSequence` interface does not guarantee an `equals` method implementation. Our LLM-based patch generator successfully generated the correct fix shown in Figure 3.

2) Lang-13: Primitive Type Serialization Issue

Bug Description: Lang-13 involves a fault in the `SerializationUtils.clone` method that fails to properly handle primitive type classes during serialization, leading to test failures in `testPrimitiveTypeClassSerialization`.

Fault Localization: The quality-aware GBMFL precisely identified the `clone` method as the primary suspect with a perfect suspicion score of 1.000.

Test Failure Analysis: The framework determined that `testPrimitiveTypeClassSerialization` was failing due to improper handling of primitive wrapper types during the serialization/deserialization process, indicating insufficient optimization for primitive types in the cloning logic.

First-Attempt Success: Lang-13 was successfully repaired in the first attempt on the highest-ranked suspicious method.

The issue was that primitive wrapper types were being unnecessarily serialized.

These case studies demonstrate complementary strengths of AutoStructor: Lang-14 showcases systematic iteration through multiple suspicious methods until finding the correct solution, while Lang-13 demonstrates the precision of quality-aware fault localization in identifying the exact problematic method on the first attempt. Both cases highlight the framework's ability to generate correct patches that address root causes rather than simply passing test cases.

```java
public static boolean equals(CharSequence cs1,
    CharSequence cs2) {
   if (cs1 == cs2) {
       return true;
   }
   if (cs1 == null || cs2 == null) {
       return false;
   }
   return cs1.toString().equals(cs2.toString());
     // FIXED: Convert to String
}
```

Fig. 3: AutoStructor Generated Fix for Lang-14

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on May 22,2026 at 23:53:41 UTC from IEEE Xplore. Restrictions apply.

```
public static <T extends Serializable> T clone(T
    object) {
   if (object == null) {
       return null;
   }
   // OPTIMIZATION: Handle primitive wrapper
    types directly
   if (object instanceof Boolean || object
    instanceof Character ||
       object instanceof Byte || object
    instanceof Short ||
       object instanceof Integer || object
    instanceof Long ||
       object instanceof Float || object
    instanceof Double) {
       return (T) object;  // Return directly -
    no serialization needed
   }
   // Continue with normal serialization for
    complex objects
   byte[] objectData = serialize(object);
   ByteArrayInputStream bais = new
    ByteArrayInputStream(objectData);
   ClassLoaderAwareObjectInputStream in = null;
   try {
       in = new ClassLoaderAwareObjectInputStream
    (bais,
               object.getClass().getClassLoader()
    );
       @SuppressWarnings("s")
       T readObject = (T) in.readObject();
       return readObject;
   } catch (ClassNotFoundException ex) {
       throw new SerializationException("s", ex);
   } catch (IOException ex) {
       throw new SerializationException("s", ex);
   } finally {
       try {
           if (in != null) {
               in.close();
           }
       } catch (IOException ex) {
           throw new SerializationException("s",
    ex);
       }
   }
}
```

Fig. 4: AutoStructor Generated Fix for Lang-13

V. Conclusion and Future Work

This paper presents AutoStructor, a novel framework that successfully addresses fundamental limitations in automated program repair by integrating Quality-Aware Graph-Based Machine Learning Fault Localization with Large Language Model-based patch generation. Our approach introduces hybrid multi-modal attention mechanisms, integration of 21 design quality metrics into fault localization, and intelligent failed test explanation bridging fault localization with LLM-based patch generation. These innovations achieve 81.9% Top-1 fault localization accuracy, Mean First Rank of 1.13, and successful repair of 48 out of 61 Lang project bugs at $0.03 total cost, representing a 50% improvement over the best existing LLM-based APR tool while maintaining remarkable cost-effectiveness through compact models.

While our evaluation demonstrates strong performance on the Defects4J Lang project, several promising research directions emerge for future work. Scaling our approach to the full Defects4J benchmark and real-world datasets like SWE-Bench would support broader validation across programming paradigms and bug complexity levels. Extending our quality-aware fault localization framework to multi-language environments entails developing language-agnostic dependency representations and adapting quality metrics to language-specific characteristics. Additionally, integrating dynamic metric weighting and fault localization confidence into patch generation, as well as exploring security-focused repair, represent compelling paths forward.

References

[1] X. Li, W. Li, Y. Zhang, and L. Zhang, "DeepFL: integrating multiple fault diagnosis dimensions for deep fault localization," in Proc. 28th ACM SIGSOFT Int. Symp. Software Testing and Analysis, Beijing, China, Jul. 15-19, 2019, pp. 169–180.

[2] Y. Lou et al., "Boosting coverage-based fault localization via graph-based representation learning," in Proc. 29th ACM Joint Meeting European Software Engineering Conf. and Symp. Foundations of Software Engineering, Athens, Greece, Aug. 23-28, 2021, pp. 664–676.

[3] Y. Li, S. Wang, and T. N. Nguyen, "Fault localization with code coverage representation learning," in Proc. 43rd Int. Conf. Software Engineering, Madrid, Spain, May 25-28, 2021, pp. 661–673.

[4] J. A. Jones and M. J. Harrold, "Empirical evaluation of the tarantula automatic fault-localization technique," in Proc. 20th IEEE/ACM Int. Conf. Automated Software Engineering, Long Beach, CA, USA, Nov. 7-11, 2005, pp. 273–282.

[5] R. Abreu, P. Zoeteweij, and A. J. van Gemund, "An evaluation of similarity coefficients for software fault localization," in Proc. 12th Pacific Rim Int. Symp. Dependable Computing, Riverside, CA, USA, Dec. 18-20, 2006, pp. 39–46.

[6] M. N. Rafi, D. J. Kim, A. R. Chen, T. H. Chen, and S. Wang, "Towards Better Graph Neural Network-based Fault Localization Through Enhanced Code Representation," arXiv preprint arXiv:2404.04496, 2024.

[7] D. Kim, J. Nam, J. Song, and S. Kim, "Automatic patch generation learned from human-written patches," in Proc. 35th Int. Conf. Software Engineering, San Francisco, CA, USA, May 18-26, 2013, pp. 802–811.

[8] Z. Chen, S. Kommrusch, M. Tufano, L. N. Pouchet, D. Poshyvanyk, and M. Monperrus, "SequenceR: Sequence-to-sequence learning for end-to-end program repair," IEEE Trans. Software Eng., vol. 47, no. 9, pp. 1943–1959, Sep. 2021.

[9] N. Jiang et al., "CURE: Code-Aware Neural Machine Translation for Automatic Program Repair," in Proc. 43rd IEEE/ACM Int. Conf. Software Engineering, Madrid, Spain, May 25-28, 2021, pp. 1161–1173.

[10] X. Yin, C. Ni, S. Wang, Z. Li, L. Zeng, and X. Yang, "ThinkRepair: Self-Directed Automated Program Repair," in Proc. 33rd ACM SIGSOFT Int. Symp. Software Testing and Analysis, Vienna, Austria, Sep. 16-20, 2024, pp. 1189–1201.

[11] A. Z. H. Yang, R. Martins, C. Le Goues, and V. J. Hellendoorn, "Large Language Models for Test-Free Fault Localization," in Proc. 46th IEEE/ACM Int. Conf. Software Engineering, Lisbon, Portugal, Apr. 14-20, 2024, pp. 1-12.

[12] T. J. McCabe, "A Complexity Measure," IEEE Trans. Software Eng., vol. SE-2, no. 4, pp. 308–320, Dec. 1976.

[13] S. R. Chidamber and C. F. Kemerer, "A Metrics Suite for Object Oriented Design," IEEE Trans. Software Eng., vol. 20, no. 6, pp. 476–493, Jun. 1994.

[14] V. R. Basili, L. C. Briand, and W. L. Melo, "A Validation of Object-Oriented Design Metrics as Quality Indicators," IEEE Trans. Software Eng., vol. 22, no. 10, pp. 751–761, Oct. 1996.

[15] F. Buzluca and C. Biray, "A Learning-Based Method for Detecting Defective Classes in Object-Oriented Systems," Software Engineering Research, vol. 23, no. 4, pp. 412–428, 2015.

[16] "AutoStructor Repository," GitHub, 2024. [Online]. Available: https://github.com/hasenoz/AutoStructor

[17] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs," in Proc. 2014 Int. Symp. Software Testing and Analysis, San Jose, CA, USA, Jul. 21-25, 2014, pp. 437–440.

[18] S. Zhang et al., "RepairAgent: An Autonomous, LLM-Based Agent for Program Repair," arXiv preprint arXiv:2403.17134, 2024.

[19] E. Jimenez et al., "How Far Can We Go with Practical Function-Level Program Repair?" arXiv preprint arXiv:2404.12833, 2024.

Authorized licensed use limited to: UNIVERSITAETSBIBL STUTTGART. Downloaded on May 22,2026 at 23:53:41 UTC from IEEE Xplore. Restrictions apply.